第2回：JDBC接続の基本〜JavaからDBへ「橋」をかける〜

1. 本日の内容の概要（最初の確認）

今日は、**Java から DB に命令を出すための入口（JDBC）**を、短い手順として固める回です。

前回の SQL（何を頼むか）に、**接続→準備→実行→結果**の型を重ねて覚えます。

Java と SQL（DB）の位置づけ：SQL は「言語」、JDBC は「Java からその言語を運ぶための橋（API）」です。

Java と DB をつなぐ JDBC の橋のイメージ

2. 学習目標（手順を覚える）

この回では、JDBC の基本パイプラインを**順番通りに言える**こと、と**安全側の書き方（? とバインド）**を掴むことを目指します。

▼ JDBC の4ステップを順番で説明できる

接続（Connection）→ SQL 準備（PreparedStatement）→ 実行（execute*）→ 結果（ResultSet）の流れを、図とあわせて説明できる。

- **Connection**：DB への接続を作る。
- **PreparedStatement**：SQL を安全に組み立てる（? に値をバインド）。
- **executeQuery / executeUpdate**：SELECT と更新系で使い分ける。
- **ResultSet**：SELECT の結果を1行ずつ読む。

▼ PreparedStatement を使う理由を一言で言える

値を文字列連結で SQL に埋め込まず、**? に後から値を渡す**ことで、SQL インジェクション（悪意ある入力の混入）を避けやすくする、という説明ができる。

▼ 手順の型（写経テンプレ）

次の並びを、自分の言葉でも追えること。

```
Connection ▶ PreparedStatement ▶ executeQuery / executeUpdate ▶ ResultSet
```

3. 写経パート（JDBC の型を体に入れる）

3-1. 用語の確認（最小）

- **JDBC**：Java から DB にアクセスするための標準 API（ドライバ経由で DBMS と会話する）。
- **Connection**：DB への接続（セッション）を表すオブジェクト。
- **PreparedStatement**：SQL を一度形にし、`?` に値を後から埋めるためのオブジェクト。
- **ResultSet**：SELECT の結果（表のイメージ）を、カーソルを進めながら読むオブジェクト。
- **ドライバ（mysql-connector-j など）**：JDBC の命令を、各 DBMS が理解できる形に変換する部品（クラスパスに追加する）。

3-2. 図でつかむ（4ステップ・安全性・読み方）

基本パイプライン（4ステップ）

JDBC の4ステップ（Connection から ResultSet）

?（プレースホルダ）とバインド：文字列連結で SQL を作らないのが基本です。

PreparedStatement のプレースホルダとバインド

ResultSet：next と get*

ResultSet の next と get メソッド

items テーブルを Java から SELECT（流れ）：前回の `items` を題材にします。

items テーブルを Java から SELECT する流れ

3-3. Java サンプル（items：category を ? で検索）

次のサンプルは、**接続情報（URL / ユーザー / パスワード）**は自分の環境に合わせて置き換えてください。DB 側に `items` がある前提です（前回の DDL を流し込んでおくとスムーズです）。

依存ライブラリ（例：**mysql-connector-j**）はプロジェクトに追加し、実行時のクラスパスに含めます。

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

public class ItemsSelectSample {

        public static void main(String[] args) {
                String url = "jdbc:mysql://localhost:3306/sample_db?useSSL=false&serverT
                String user = "root";
                String password = "password";

                String sql = "SELECT item_id, category, item_name, unit_price FROM items

                try (
                        Connection conn = DriverManager.getConnection(url, user, passwor
                        PreparedStatement ps = conn.prepareStatement(sql);
                ) {
                        ps.setString(1, "drink");

                        try (ResultSet rs = ps.executeQuery()) {
                                while (rs.next()) {
                                        int itemId = rs.getInt("item_id");
                                        String category = rs.getString("category");
                                        String itemName = rs.getString("item_name");
                                        int unitPrice = rs.getInt("unit_price");

                                        System.out.println(itemId + " | " + category + "
                                }
                        }
                } catch (SQLException e) {
                        e.printStackTrace();
                }
        }
}
```

▼ サンプルコードをドラッグでコピー（クリックで開く）

下のプレーンテキストの貼り付け欄から、Eclipse などへドラッグで取り込めます。

マウスで全選択して貼り付け欄へドラッグしてください：

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

public class ItemsSelectSample {
```

4. 練習問題（JDBC）

次の問いに、手元のメモ用紙か、SQL ツール／IDE の実行結果で答えてください。まず自分で考えてから、ヒントや解答を確認してください。

問題1：JDBC の基本パイプラインを、**4つの要素名**を **実行順**に並べて書いてください（英語のクラス名でOK）。

▼ ヒント

「つなぐ → SQL準備 → 実行 → 結果を読む」の順です。SELECT の結果を最後に扱うのはどのオブジェクトでしたか。

▼ 解答例

```
1) Connection
2) PreparedStatement
3) executeQuery / executeUpdate（実行）
4) ResultSet（SELECT のとき）
```

マウスで全選択して貼り付け欄へドラッグしてください：

```
1) Connection
2) PreparedStatement
3) executeQuery / executeUpdate（実行）
4) ResultSet（SELECT のとき）
```

問題2：`SELECT` を実行するときに使うメソッド名を書いてください。

▼ ヒント

結果セット（表のイメージ）を返すメソッドです。

▼ 解答例

```
executeQuery()
```

マウスで全選択して貼り付け欄へドラッグしてください：

```
executeQuery()
```

問題3：`INSERT` / `UPDATE` / `DELETE` を実行するときに使うメソッド名を書いてください。

▼ ヒント

更新件数（int）を返す系の実行メソッドです。

▼ 解答例

```
executeUpdate()
```

マウスで全選択して貼り付け欄へドラッグしてください：

```
executeUpdate()
```

問題4：`ResultSet` で「次の行」に進むメソッド名を書いてください。

▼ ヒント

`while` ループとセットでよく書きます。

▼ 解答例

```
next()
```

マウスで全選択して貼り付け欄へドラッグしてください：

```
next()
```

問題5：`?` に値を入れる処理を、教科書的な用語で何と呼ぶか書いてください（一般的な日本語名でOK）。

▼ ヒント

`setString` / `setInt` などの前段の話です。

▼ 解答例

バインド（パラメータバインド）

マウスで全選択して貼り付け欄へドラッグしてください：

バインド（パラメータバインド）

問題6：ユーザー入力を文字列連結で SQL に埋め込まず、`PreparedStatement` を使う主な理由を、セキュリティ観点で一言で書いてください。

▼ ヒント

「入力が SQL の形を壊して、意図しない命令が混ざる」系の攻撃を、設計上しづらくします。

▼ 解答例

SQL インジェクション対策（悪意ある入力を SQL として解釈されにくくする）

マウスで全選択して貼り付け欄へドラッグしてください：

SQL インジェクション対策（悪意ある入力を SQL として解釈されにくくする）

5. ラボ（10分チェックリスト）

10分で終わる最小確認（講師のOKが出るまで次に進めない、の運用にしてください）

1. JDBC URL（ホスト名・ポート・DB名）が自分の環境と一致している。
2. MySQL ドライバ（例：mysql-connector-j）がプロジェクトに入り、実行できる。
3. `SELECT` が `executeQuery` で1回通り、コンソールに行が出る。
4. `?` のバインド（例：`setString(1, "drink")`）を1か所変えて、結果が変わることを確認する。
5. 例外が出たら、スタックトレースの先頭メッセージを読み、接続／SQL／権限のどれかに切り分ける。
6. 余力がある人だけ：`INSERT` を `executeUpdate` で1回通し、次の `SELECT` で増えたことを確認する。

6. まとめ（写経で締める）

```
1) Connection：DB に接続する
2) PreparedStatement：SQL を準備し、? に値をバインドする
3) 実行：SELECT は executeQuery／更新は executeUpdate
4) ResultSet：SELECT の結果を next と get* で読む
```